



Universidade do Minho
Escola de Engenharia

Sistemas Operativos

Trabalho Prático

Grupo 68 LEI (2ºano)

Guilherme Meireles Baptista	A111906
Martim Amaral Silva Monteiro Rodrigues	A107320
Matteo Rodrigues Murari	A111255

1. Introdução

O presente relatório descreve a concepção e implementação de um sistema de orquestração de execução de comandos *bash*, desenvolvido no âmbito da unidade curricular de Sistemas Operativos. O sistema é composto por dois programas distintos, *runner* e *controller*, que cooperam através de mecanismos de comunicação interprocesso para permitir a execução concorrente de comandos submetidos por múltiplos utilizadores.

O *runner* é o programa com que o utilizador interage diretamente. A sua função é notificar o *controller* da intenção de executar um comando e aguardar autorização antes de o fazer. O *controller*, por sua vez, é um servidor que recebe pedidos de múltiplos *runners* em simultâneo, os escalona de acordo com uma política configurável, e notifica cada *runner* quando a execução pode iniciar. Importa sublinhar que o *controller* nunca executa comandos diretamente, a execução é sempre da responsabilidade do *runner* o qual não é um programa interativo.

2. Arquitectura de Processos

2.1 Visão geral

O sistema segue uma arquitetura cliente-servidor assente em processos independentes. O *controller* é o servidor, arranca uma única vez, cria um *pipe* com nome público e entra num ciclo bloqueante à espera de mensagens. Os *runners* são os clientes, cada invocação do programa *runner* é um processo independente que comunica com o *controller*, executa a sua tarefa, e termina.

Esta separação de responsabilidades é deliberada. O *controller* nunca bloqueia à espera que um comando termine, essa espera é feita pelo *runner* via `waitpid()`. O *controller* limita-se a gerir quem tem permissão para executar e quando, mantendo-se sempre disponível para receber novos pedidos de outros *runners*. Este modelo garante que o processamento de um pedido de consulta (-c) ou de desligar (-s) não é bloqueado pela execução de um comando longo. Abaixo encontra-se um resumo da arquitetura de processos com as diferentes opções de execução do *runner*:

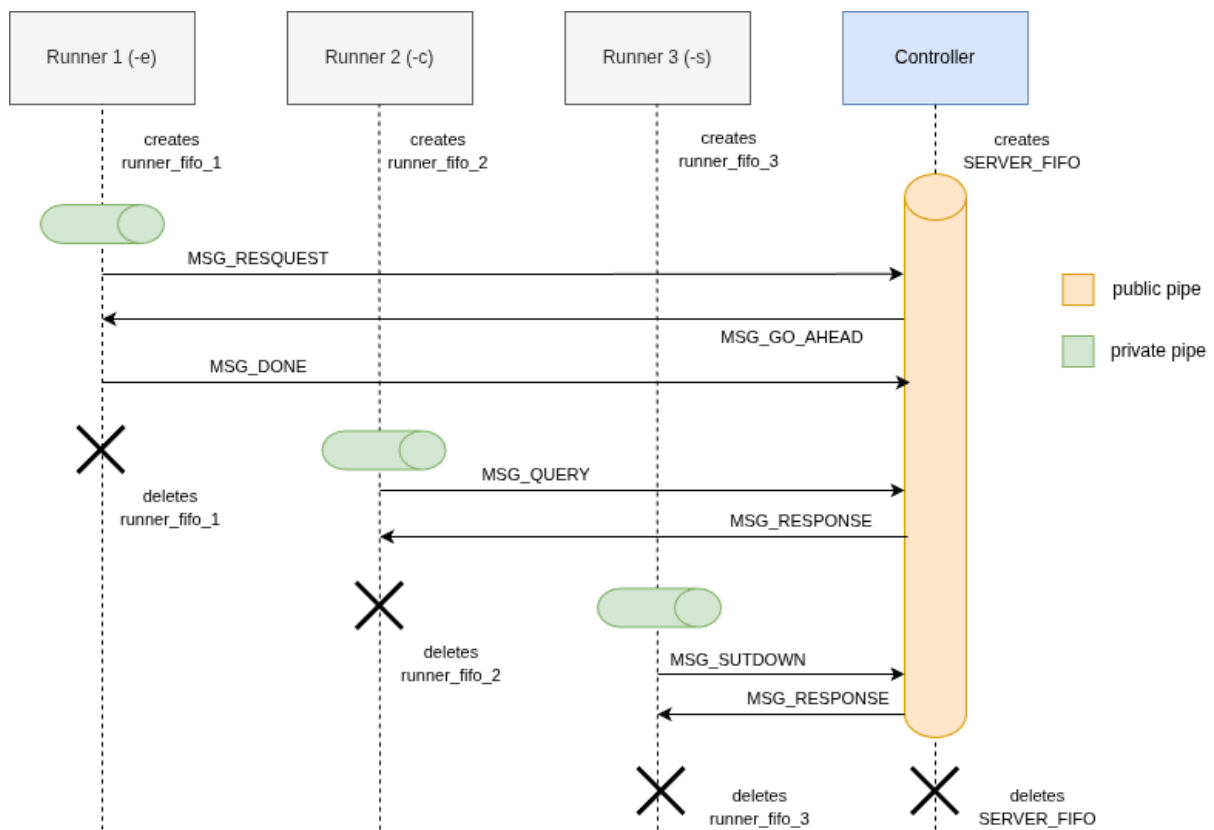


Figura 1: Resumo da arquitetura de processos

2.2 Ciclo de vida de um runner

Cada invocação do runner com a opção `-e` segue o seguinte ciclo:

1. Cria um *pipe* com nome privado em `tmp/runner_fifo_<pid>`, onde `<pid>` é o identificador do processo obtido via `getpid()`. A utilização do PID garante unicidade, dois runners a correr em simultâneo nunca partilham o mesmo *pipe* privado.
2. Envia uma mensagem do tipo `MSG_REQUEST` ao controller através do *pipe* público.
3. Bloqueia em `read()` no seu *pipe* privado, aguardando autorização.
4. Ao receber `MSG_GO_AHEAD`, executa o comando via `fork()` + `execvp()` e aguarda a conclusão do processo filho com `waitpid()`.
5. Envia `MSG_DONE` ao controller e termina, apagando o *pipe* privado com `unlink()`.

O bloqueio no passo 3 é intencional e não representa consumo de CPU, o *kernel* suspende o processo e só o retoma quando há dados disponíveis no *pipe*, como descrito na seção seguinte.

2.3 Ciclo de vida do controller

O controller arranca, valida os argumentos `<parallel-commands>` e `<sched-policy>`, inicializa o *scheduler* com a política indicada por argumento, cria o *pipe* público e abre-o em modo

O_RDWR. Entra depois num ciclo onde bloqueia em `read()` à espera de mensagens. Para cada mensagem recebida, delega o processamento ao *handler* correspondente e regressa imediatamente ao ciclo.

A abertura do *pipe* público em modo O_RDWR em vez de O_RDONLY é uma decisão de desenho com consequências práticas importantes. Com O_RDONLY, o `read()` devolve zero (EOF) quando não existe nenhum runner com o *pipe* aberto para escrita, o que quebraria o ciclo principal. Com O_RDWR, o próprio controller mantém sempre um descritor de escrita aberto sobre o mesmo *pipe*, pelo que o `read()` bloqueia indefinidamente à espera de dados em vez de devolver EOF. O processo permanece no estado SLEEPING durante este bloqueio, não consome CPU, e é acordado pelo *kernel* apenas quando um runner escreve no *pipe*.

A terminação do controller só ocorre depois de verificadas duas condições em simultâneo: todos os *slots* de execução estão livres (`all_slots_free()`) e a fila do scheduler está vazia (`scheduler_is_empty()`). Esta verificação é realizada após cada MSG_DONE recebido e também imediatamente após o MSG_SHUTDOWN, caso a fila já esteja vazia no momento do pedido.

3. Mecanismos de Comunicação Interprocesso

3.1 Protocolo de mensagens

Toda a comunicação entre runner e controller é serializada numa struct de tamanho fixo, definida em `include/common/protocol.h`:

```
typedef struct {
    MsgType type;
    pid_t runner_pid;
    int user_id;
    int job_id;
    char command[MAX_CMD];
} Message;
```

Figura 2: Struct principal para a camada ipc

A utilização de uma estrutura de tamanho fixo é fundamental para a correta delimitação de mensagens em *pipes*. Uma escrita de `sizeof(Message)` bytes é lida atomicamente do outro lado como exatamente um bloco, não existe ambiguidade sobre onde uma mensagem termina e a seguinte começa, ao contrário do que aconteceria com mensagens de comprimento variável, que exigiriam um mecanismo explícito de delimitação.

3.2 Tipos de mensagem e transições de estado

O protocolo define seis tipos de mensagem. A tabela seguinte resume a direcção e o propósito de cada um, bem como a resposta que o receptor deve produzir:

Tipo	Direcção	Propósito	Resposta esperada
MSG_REQUEST	runner → controller	Pedir autorização	MSG_GO_AHEAD
MSG_GO_AHEAD	controller → runner	Conceder autorização	Execução do comando
MSG_DONE	runner → controller	Notificar conclusão	Liberação do <i>slot</i>
MSG_QUERY	runner → controller	Pedir listagem (-c)	MSG_RESPONSE
MSG_RESPONSE	controller → runner	Entregar listagem	Impressão em stdout
MSG_SHUTDOWN	runner → controller	Pedir terminação (-s)	MSG_RESPONSE

Tabela 1: Sumário dos tipos de mensagens que existem na comunicação interprocesso

As transições de estado do *controller* induzidas por estas mensagens podem ser vistas como um autómato: MSG_REQUEST move um *job* do estado “submetido” para “agendado” ou imediatamente para “em execução”; MSG_DONE move o *job* para “concluído” e potencialmente move o próximo da fila para “em execução”; MSG_SHUTDOWN move o servidor para um estado de “terminação pendente” que só se concretiza quando a fila estiver vazia.

4. Políticas de Escalonamento

4.1 Decisão de Design

Foram implementadas duas políticas de escalonamento seleccionáveis por argumento na linha de comandos: FCFS e Round Robin. Ambas partilham a mesma interface pública definida em `include/common/scheduler.h`, sendo a selecção feita internamente por um campo `policy` na estrutura `Scheduler`, que reencaminha cada chamada para a implementação correcta. Esta decisão permite ao *controller* invocar sempre a mesma interface sem conhecer a política activa, e permite adicionar novas políticas no futuro sem alterar o *controller*.

A escolha de implementar duas políticas distintas responde diretamente ao requisito do enunciado de garantir justiça entre utilizadores, o FCFS é suficiente para cargas homogéneas, enquanto o RR

garante que nenhum utilizador é sistematicamente prejudicado em cargas heterogéneas. A selecção da política mais adequada fica assim ao critério do operador do sistema.

4.2 First-Come, First-Served (FCFS)

A política FCFS é implementada como uma lista ligada simples com dois ponteiros, *head* e *tail*, definidos na estrutura *FCFSData*. Como ilustrado na Figura 3, *head* aponta para o primeiro *job* a ser despachado e *tail* aponta para o último a ter chegado, a inserção é feita sempre no fim e a remoção sempre pela frente, ambas em tempo constante.

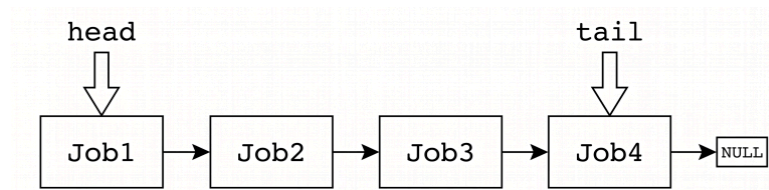


Figura 3: Representação visual da struct *FCFSData* com 4 *jobs*

O comportamento desta política é determinístico: os *jobs* são despachados exatamente pela ordem em que chegaram ao *controller*. Esta propriedade é adequada quando todos os utilizadores submetem *jobs* com durações semelhantes, dado que a ordem de chegada é uma aproximação razoável de justiça. O problema surge em cargas heterogéneas, se um utilizador submete *jobs* longos antes de outro submeter *jobs* curtos, estes últimos ficam sistematicamente encostados atrás dos primeiros sem qualquer hipótese de avançar, fenómeno designado efeito *convoy*. É precisamente para resolver esta limitação que o Round Robin foi implementado.

4.3 Round Robin (RR)

A política Round Robin é implementada como uma lista circular de filas por utilizador. Cada nó da lista é uma *UserQueue*, uma fila FCFS interna que contém todos os *jobs* pendentes de um dado utilizador. As *UserQueue* estão ligadas em círculo e um ponteiro *current* indica qual o utilizador cujo próximo *job* será despachado. Como ilustrado na Figura 4, a estrutura mantém em simultâneo a ordem de chegada dos *jobs* dentro de cada utilizador e a rotação justa entre utilizadores distintos.

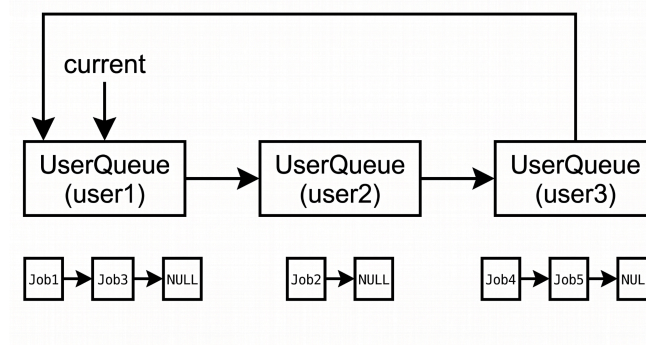


Figura 4: Representação visual da Struct *UserQue* com 3 utilizadores e 5 *jobs*

Importa distinguir esta implementação do RR clássico das aulas: enquanto o modelo clássico usa um *quantum* de tempo de CPU como unidade de escalonamento, interrompendo o processo quando esse *quantum* expira, o *quantum* aqui é o *job* completo; desta forma, cada utilizador recebe exactamente um *slot* por volta, independentemente de quantos *jobs* tenha pendentes. A alternância entre utilizadores ocorre exclusivamente quando um *slot* fica livre, o controller não tem mecanismo para interromper um comando a meio, pois a execução é feita pelo runner e é opaca ao controller. A cada despacho, o *scheduler* retira o *job* da frente da fila do utilizador apontado por *current* e avança imediatamente o ponteiro para o nó seguinte. Quando um utilizador esgota todos os seus *jobs*, o respectivo nó é removido do círculo para que *current* nunca itere sobre entradas vazias.

Esta estrutura resolve diretamente o efeito *convoy* do FCFS: um utilizador com 10 jobs pendentes não bloqueia os restantes. Com três utilizadores activos, a sequência de despacho segue a rotação `user1 → user2 → user3 → user1 → . . .`, pelo que nenhum utilizador espera mais do que $(n-1)$ despachos entre dois dos seus *jobs* consecutivos, onde n é o número de utilizadores activos. A justiça é assim garantida ao nível do utilizador, e não apenas ao nível do job individual como no FCFS.

4.4 Identificação dos jobs

O identificador de cada job é gerado pelo runner antes de qualquer comunicação com o controller, combinando o PID do processo com um contador estático crescente. O PID garante unicidade entre runners distintos em execução simultânea, e o contador diferencia múltiplos comandos submetidos pela mesma instância do runner. Este identificador é colocado no campo `job_id` da Message e enviado no MSG_REQUEST, o controller recebe-o e passa-o diretamente ao *scheduler*, sem gerar um identificador próprio. O resultado prático é que o identificador apresentado pelo runner no output do `-e` e o identificador listado pelo `-c` são sempre o mesmo valor, tornando trivial para o utilizador correlacionar um comando submetido com o seu estado na fila.

5. Paralelismo

O controller suporta a execução simultânea de até `<parallel>` comandos, valor recebido como argumento na linha de comandos. Este grau de paralelismo é gerido através de um *array* de Structs `RunningSlot` com dimensão `MAX_PARALLEL`, declarado na *main* e inicializado com todos os campos `active` a zero. A Figura 5 representa este *array*: cada célula corresponde a um *slot* com um campo `active` que indica se está ocupado, e os campos `job_id`, `user_id`, `runner_pid` e `command` que identificam o *job* em execução. O *array* é necessário porque, quando um *job* é despachado, a sua estrutura *Job* é libertada do *scheduler*, o controller precisa de continuar a saber quem está a executar para responder ao `-c` e para calcular a duração do job quando o MSG_DONE chegar.

slot 0	slot 1	slot 2	slot 3
active=1 job_id=15603 user_id=1	active=0	active=0	active=0

Figura 5: Representação do *array* de Struct *RunningSlot* com um job ativo

A gestão do *array* assenta em duas funções auxiliares. A função *find_free_slot* percorre o *array* e devolve o índice do primeiro *slot* com *active == 0*, ou -1 se todos estiverem ocupados. É chamada em *handle_request* quando chega um *MSG_REQUEST*: se existir um *slot* livre, o *job* é imediatamente despachado via *dispatch_next* para esse *slot*; caso contrário, entra na fila do *scheduler*. Quando um *MSG_DONE* é recebido, o *slot* correspondente é marcado como livre e *dispatch_next* é chamado imediatamente, se houver *jobs* em espera, o próximo é despachado para o *slot* que acabou de libertar. Com *parallel=2* e três *jobs* submetidos em simultâneo, por exemplo, os dois primeiros ocupam os *slots* 0 e 1 e o terceiro aguarda na fila; quando qualquer um dos dois termina, o terceiro é despachado sem esperar pelo outro.

6. Avaliação Experimental

6.1 Metodologia

Para validar a eficácia do escalonador e o impacto do paralelismo, foram utilizadas as métricas clássicas de gestão de processos:

- *Turnaround Time*: Tempo total que o utilizador espera, desde a submissão até ao fim da execução, calculado via $T_{\text{turnaround}} = T_{\text{completion}} - T_{\text{arrival}}$.
- *Response Time*: Rapidez com que o sistema reage ao pedido, calculado por $T_{\text{response}} = T_{\text{firstrun}} - T_{\text{arrival}}$.
- *Throughput*: Número de comandos finalizados por unidade de tempo.
- *Fairness* (Justiça): Analisada através do desvio padrão do turnaround médio entre diferentes utilizadores, um valor mais baixo indica distribuição mais equitativa do tempo de espera.

Os testes foram executados através de um *script bash* e de um programa Python de geração de gráficos, ambos produzidos com o auxílio da ferramenta Claude AI. O *prompt* utilizado na geração destes ficheiros encontra-se em anexo. Foram definidos dois cenários de teste controlados:

1. Cenário A (Homogéneo): 3 utilizadores, cada um com 10 comandos idênticos (*sleep 2*). Serve de cenário de controlo para validar o comportamento base com carga uniforme.

2. Cenário B (Heterogéneo): Utilizador 1 com *jobs* de 5s, Utilizador 2 com *jobs* de 2s, Utilizador 3 com *jobs* de 0,2s. Visa expor o efeito *convoy* do FCFS e a vantagem de justiça do RR em cargas mistas.

Cada cenário foi testado com paralelismo de 1, 2 e 10 *slots*, com as políticas FCFS e RR.

6.2 Resultados e Discussão

6.2.1. Cenário A — Homogéneo

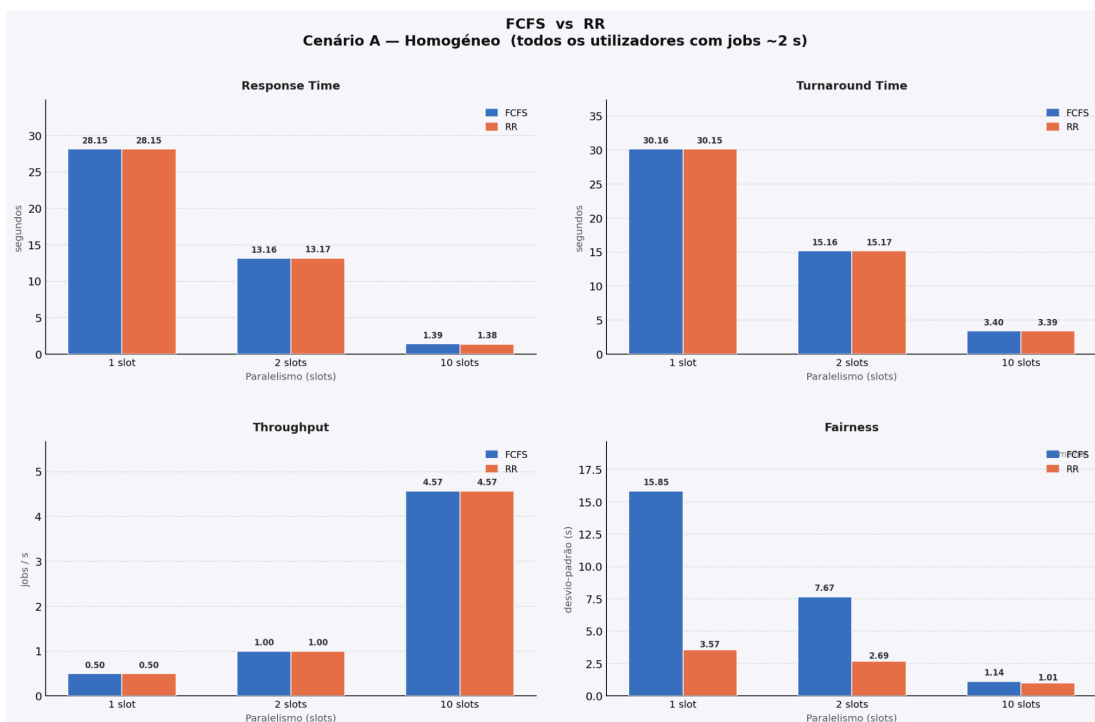


Figura 6: Métricas do Cenário A para FCFS e RR com paralelismo de 1, 2 e 10 *slots*.

Com carga uniforme, FCFS e RR apresentam desempenho praticamente indistinguível em todas as métricas. Com 1 *slot*, ambas as políticas registam um *Response Time* de 28,15s e um *Turnaround* médio de 30,16s, valores elevados que refletem a execução sequencial de 30 *jobs*, em que o último *job* espera a conclusão de todos os anteriores. O resultado confirma que, sem disparidade na carga, a ordem de chegada do FCFS é naturalmente justa e o RR não introduz qualquer vantagem.

O aumento do paralelismo tem um impacto determinante: de 1 para 10 *slots*, o *Turnaround* médio desce de 30,16s para 3,40s e o *Throughput* aumenta de 0,50 para 4,57 *jobs/s*. Esta escalabilidade é independente da política, em cargas homogéneas, o paralelismo é o factor dominante no desempenho do sistema.

6.2.2 Cenário B — Heterogéneo

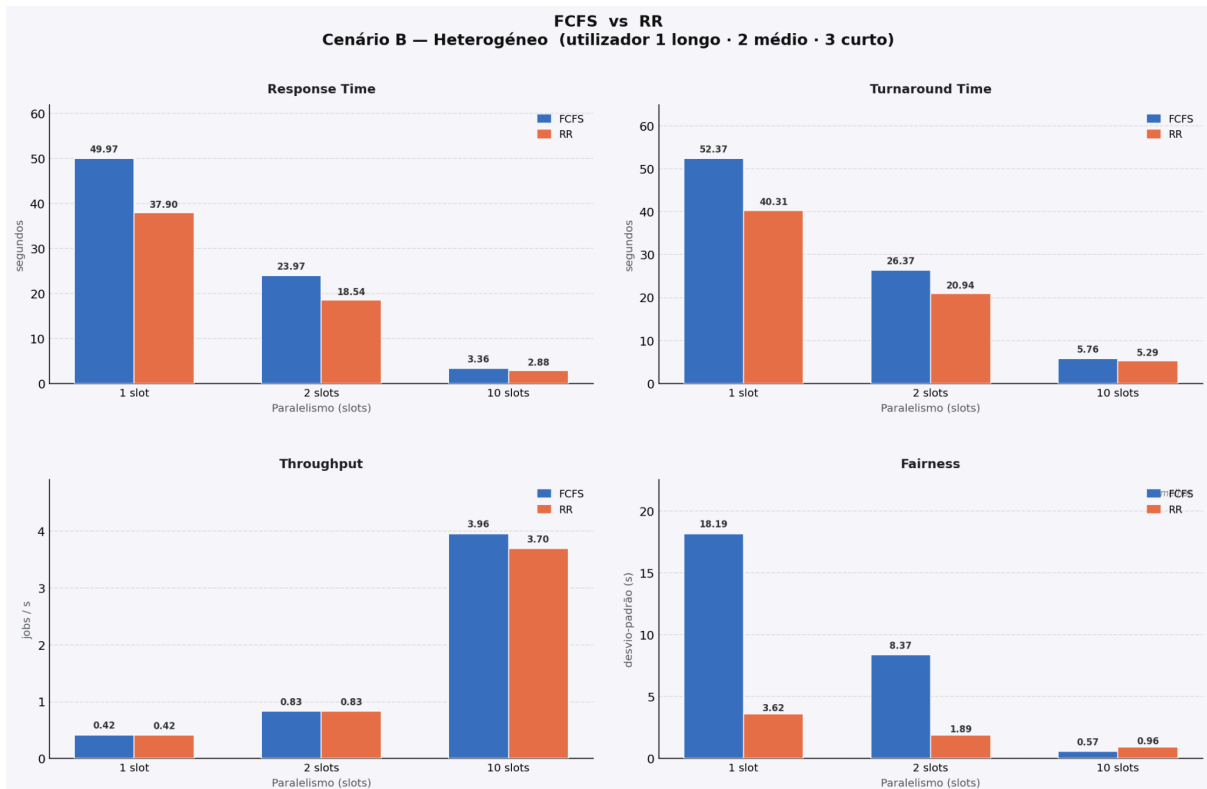


Figura 7: Métricas do Cenário B para FCFS e RR com paralelismo de 1, 2 e 10 slots.

Este cenário expõe a principal limitação do FCFS. Com 1 *slot*, o desvio padrão do *turnaround* entre utilizadores é de 18,19s no FCFS e de 3,62s no RR, uma redução de cerca de 80% que evidencia a distribuição mais equitativa do tempo de espera proporcionada pelo Round Robin.

O efeito *convoy* é observável diretamente nos dados por utilizador: no FCFS, o Utilizador 3 (*jobs* de 0,2s) regista um *turnaround* médio de 69,69s, sendo sistematicamente arrastado pelas tarefas longas dos outros utilizadores que chegaram primeiro à fila. No RR, este valor desce para 42,65s, porque o *scheduler* alterna entre utilizadores a cada *slot* libertado, permitindo que as tarefas curtas concluam sem aguardar a execução completa dos *jobs* longos.

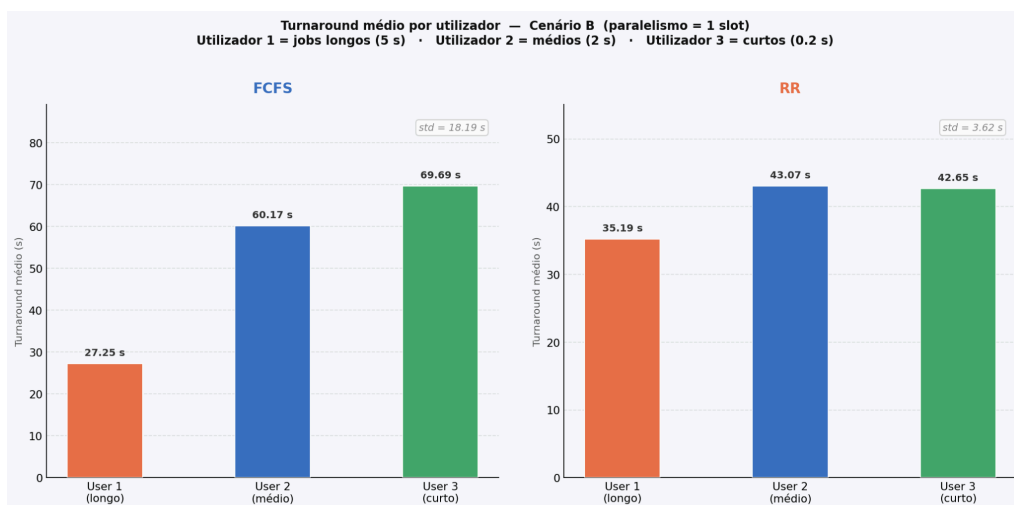


Figura 8: *Turnaround* médio por utilizador no Cenário B com paralelismo de 1 slot.

6.2.3 Trade-offs e Impacto do Paralelismo

O RR é consistentemente mais justo, mas o FCFS apresenta um *Throughput* ligeiramente superior em cenários de alto paralelismo, 3,96 vs 3,70 *jobs/s* com 10 *slots* no Cenário B. Esta diferença explica-se pelo facto de o FCFS, ao não alternar entre utilizadores, e tender a despachar *jobs* do mesmo utilizador em sequência, aproveitando melhor a localidade de submissão. Ainda assim, a melhoria expressiva no tempo de resposta para utilizadores com tarefas curtas justifica a adopção do RR em cargas heterogéneas, onde a justiça entre utilizadores é o critério prioritário.

O paralelismo revela-se o factor com maior impacto absoluto no desempenho: independentemente da política, o aumento de 1 para 10 *slots* reduz o *turnaround* e aumenta o *throughput* de forma proporcional em ambos os cenários. Importa ainda notar que nenhuma das políticas implementadas é preemptiva, uma vez despachado, um *job* não pode ser interrompido pelo controller. No FCFS, esta ausência de preempção é particularmente penalizante com 1 *slot*, onde um utilizador com *jobs* longos ocupa o único *slot* disponível sem que o *scheduler* possa ceder o controlo a outro utilizador. O aumento do paralelismo mitiga esta limitação ao disponibilizar *slots* adicionais, aproximando o comportamento das duas políticas à medida que a probabilidade de um *slot* livre aumenta

7. Conclusão

O trabalho permitiu a implementação de um sistema de orquestração de execução de comandos *bash* em Linux, assente numa arquitetura cliente-servidor comunicante através de *named pipes* com mensagens de tamanho fixo. Esta escolha revelou-se robusta, a atomicidade das leituras e escritas de blocos de tamanho fixo elimina ambiguidades na delimitação de mensagens e evita condições de corrida na comunicação entre runners e controller. A implementação de duas políticas de escalonamento, FCFS e Round Robin, permitiu constatar experimentalmente que a justiça entre utilizadores e a eficiência global são objetivos em tensão: o RR mitiga o efeito *convoy* do FCFS em cargas heterogéneas à custa de um *throughput* ligeiramente inferior, enquanto o aumento do paralelismo se revelou o factor com maior impacto absoluto no desempenho independentemente da política activa.

Como perspectiva de trabalho futuro, identificam-se três melhorias concretas. A primeira é a configuração do *quantum* do Round Robin - actualmente fixo em um *job* por utilizador por volta - para um valor definido pelo operador na linha de comandos, permitindo testar o comportamento do sistema com *quantums* de dois, três ou mais *jobs* e aproximar ou afastar o RR do comportamento do FCFS de forma controlada. A segunda é a notificação ao runner quando todos os *slots* estão ocupados, actualmente o runner bloqueia silenciosamente à espera de autorização, sem qualquer *feedback* ao utilizador sobre a sua posição na fila. A terceira é a implementação de *preemption* por *timeout*, que permitiria ao controller recuperar um *slot* ocupado por um comando que exceda um limite de tempo configurável, evitando que um processo mal concebido bloqueie o sistema indefinidamente.

8. Anexos

Anexo A — *Prompt* utilizado na geração do *script* de teste

Os ficheiros `teste.sh` e `graficos.py` utilizados na avaliação experimental foram gerados com o auxílio da ferramenta Claude AI. Apresenta-se abaixo o *prompt* submetido para a sua criação:

(contexto : zip do código e o enunciado o trabalho)

Quero que me crie dois ficheiros de testes para avaliar e comparar a performance das duas políticas de escalonamento do meu código. `teste.sh`: script `bash` que corra automaticamente todas as combinações de política (`fcfs`, `rr`) × paralelismo (1, 2, 10 slots) para dois cenários. Para cada combinação: limpar logs anteriores, arrancar o controller em background, esperar que o `fifo` do `controlLer` exista, submeter os `runners` em background guardando os seus PIDs num array, fazer `wait` só nos PIDs dos `runners`, enviar `./runner -s` e guardar o log em `testes/dados/` com nome descritivo. Os dois cenários são: Cenário A (homogéneo): 3 utilizadores × 10 jobs com `sleep 2` cada, serve de baseline para quando a carga é igual para todos. Cenário B (heterogéneo): utilizador 1 com `sleep 5` (longo), utilizador 2 com `sleep 2` (médio), utilizador 3 com `sleep 0.2` (curto). `graficos.py`: script Python que leia os logs de `testes/dados/` e calcule: Response Time (`first_run - arrival`), Turnaround Time (`duration`), Throughput (`jobs / duração total da sessão`) e Fairness (desvio-padrão do turnaround médio por utilizador — quanto maior, mais injusto). Gráficos de barras agrupadas FCFS vs RR, um grupo por paralelismo, guardados em `testes/graficos/`: um gráfico com o turnaround por utilizador individualmente para o cenário B com paralelismo=1, e um comparativo de fairness entre os dois cenários.